

BRL Architecture

Changmin Park*, Seongah Park*, Nayoung Kim*, Sungjin Park*, Geonyeong Ko*, Junho Lee†,
and Changjoo Nam†

*Sogang University

cmp9877@gmail.com, ppseongah@gmail.com †Institution2
{2, 3}@def.com

Abstract—In this paper, we present a knowledge-based POMDP framework that enables robots to selectively request human feedback under uncertainty. While modern robotic systems achieve strong autonomy, they still fail due to perception errors, execution failures, and environmental changes, often requiring continuous human supervision. We model decision making as a POMDP over a symbolic knowledge base, where uncertainty is quantified using the entropy of a belief over possible world states to determine when to query. The system also determines what to query by selecting informative queries that reduce ambiguity between competing hypotheses. We evaluate the proposed method in both a simulated waste sorting task and a real-world tomato harvesting task. Results show that our approach maintains task success rates comparable to full supervision while significantly reducing the number of queries, and user studies confirm reduced cognitive workload during interaction.

Index Terms—robotic system, planning under uncertainty, human-robot interaction

I. INTRODUCTION

Recent advances in robotic systems have significantly improved autonomy in perception, manipulation, and planning across domains such as agriculture and household environments. Despite these advances, robots still encounter perception errors [1], action execution failures [2], and unexpected environmental changes [3]. As a result, many robotic systems continue to rely on human operators to monitor execution and intervene when failures occur [4], [5]. However, requiring humans to continuously monitor robotic systems and determine whether robot decisions are correct is impractical in long-horizon tasks because it increases cognitive workload and fatigue [6]. Instead of relying on human operators to detect failures after they occur, robotic systems should be able to recognize uncertain situations during execution and request assistance only when additional information is required [7], [8]. This raises two fundamental questions: (i) when should a robot request human assistance, and (ii) what information should it request?

Recent studies have explored uncertainty-aware robotic systems that selectively request human feedback during task execution [?], [9], [10]. These approaches demonstrate that uncertainty-aware querying can reduce unnecessary human intervention and improve decision making under uncertainty. However, uncertainty estimation, query generation, and planning are often treated as separate components. As a result, the relationship between when to request human assistance,

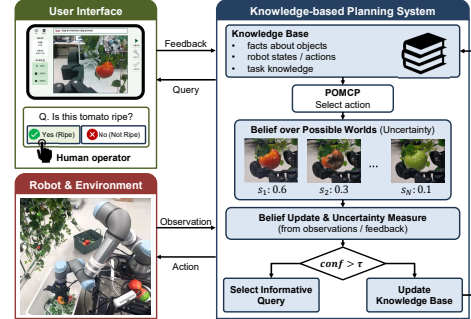


Fig. 1: Overview of the proposed knowledge-based robotic system with uncertainty-aware planning. The system maintains a belief over possible worlds and updates it based on observations. If the uncertainty exceeds a threshold, the system selectively queries the user to resolve ambiguity. Otherwise, it continues autonomous planning and execution.

what information to request, and how the acquired feedback should influence subsequent decision making remains insufficiently explored in robotic systems operating under uncertainty.

To address this challenge, we propose a knowledge-based robotic framework that integrates uncertainty estimation, query selection, and task planning into a unified decision-making process. We formulate decision making as a partially observable Markov decision process (POMDP) [2] and represent task knowledge using a STRIPS-style symbolic model [11], [12]. The system maintains a belief over alternative world hypotheses and continuously updates this belief using observations collected during execution. The resulting belief distribution is used to determine whether additional information is required. When uncertainty exceeds a threshold, the robot requests feedback that is expected to maximally reduce ambiguity among competing hypotheses. Otherwise, the robot continues autonomous planning and execution.

To support autonomous operation under uncertainty, we integrate the proposed framework into a robotic system architecture. As illustrated in Fig. 1, a centralized knowledge base connects the sensing, planning, feedback management, and user interface modules. Sensor observations and user feedback are stored in the knowledge base and shared across modules, providing a consistent representation of the environment throughout execution. Planning is performed over symbolic world states, while belief updates maintain uncertainty over alternative hypotheses. Consequently, the robot can continue

autonomous execution and request human feedback only when necessary.

We evaluate the proposed approach in two robotic domains, tomato harvesting and waste sorting. Experimental results demonstrate that the framework improves task performance while reducing the amount of human intervention required during execution. We further analyze how different query policies influence situation awareness and perceived workload through user studies.

The main contributions of this work are as follows:

- We propose an uncertainty-aware feedback triggering method for determining when human assistance is required during task execution.
- We propose an information-driven query selection method for identifying what information should be requested to reduce uncertainty over competing world hypotheses.
- We incorporate uncertainty-aware querying into the planning and execution process, enabling autonomous robot operation with selective human feedback.
- We validate the proposed approach in tomato harvesting and waste sorting tasks through robot experiments and user studies.

II. RELATED WORK

A. Knowledge-based Robotic System

In robotics, a knowledge base refers to a structured representation of information required for decision making. These systems use logic-based representations of objects, attributes, and relations to describe the world in a way that is understandable not only by robots but also by users [13], [14]. In particular, predicate-based symbolic representations such as [12], [15] allow the system to describe both object properties and relationships among objects. In addition, graph-based models such as [16], [17] are often used to represent these relationships more clearly by organizing objects and their connections in a simple structured form.

To systematically define robot states and actions, planning languages such as PDDL [15] or FDR [18] are widely adopted. In these representations, a state is defined as a set of predicates, and each action is described by its preconditions and effects, which specify how the state changes after execution. This formulation provides a clear model of state transitions and allows robots to plan over a well-defined symbolic state space. As a result, classical planning techniques [19]–[21] can be applied to compute action sequences that achieve a given goal.

Knowledge-based robotic systems store observed facts and infer new knowledge from existing information within this symbolic representation. Logical and relational inference allows robots to identify facts that are not directly observed, combine multiple relations, and obtain plausible hidden states from contextual knowledge [22], [23]. For example, a robot can infer the likely location of an unseen object from qualitative spatial relations, container-location relations, or place-object co-occurrence knowledge [24], [25]. Commonsense reasoning connects symbolic knowledge with perceptual evidence

and supports decision making in partially observable environments [26]–[28]. Ontology structures and type hierarchies further support consistent integration of new observations and generalization across related objects, actions, and task contexts [29]–[31].

While symbolic knowledge-based systems provide structured representations and support logical inference, recent approaches have explored the use of large language models (LLMs) to further expand how knowledge is utilized in robotic planning. Early LLM-based approaches employ procedural and commonsense knowledge encoded in LLMs to propose task decompositions or action sequences. The proposed plans are then grounded with robot affordances, programmatic interfaces, or execution feedback [32]–[36]. Recent LLM-based approaches also incorporate external knowledge sources to compensate for incomplete observations. Some methods use scene graphs or open-vocabulary spatial representations to provide structured information about objects, places, and relations [37], [38]. Other methods retrieve demonstrations, procedural knowledge, or task-relevant constraints to guide long-horizon planning [39]–[43]. Related methods combine LLM-generated plans with symbolic constraints, spatial reasoning, or verification mechanisms to improve executability and consistency [44]–[48].

However, many knowledge-based robotic systems still rely on deterministic symbolic representations and assume a single consistent world state, which limits their ability to represent multiple hypotheses under uncertainty [49], [50]. As a result, such systems do not explicitly model uncertainty in action effects, sensing, or world states, and inferred facts are typically committed as definite knowledge. Recent LLM-based planning methods also suffer from grounding errors, incomplete retrieval, and hallucinated inferences, and do not maintain structured uncertainty over alternative states. While knowledge-based systems provide rich semantic structure for planning, they do not directly address uncertainty in observations, inferred facts, and knowledge validity.

To address this limitation, we maintain multiple symbolic hypotheses over possible worlds instead of committing inferred facts as deterministic knowledge. Our framework combines symbolic knowledge-based reasoning with planning under uncertainty, allowing the robot to reason over uncertain inferred states while selectively requesting human feedback if the uncertainty becomes high.

B. Planning under Uncertainty

1) *POMDP*: Unlike deterministic symbolic knowledge-based systems that assume a single world state, POMDPs explicitly reason over uncertainty through belief representations over multiple possible states. In robotic environments, such uncertainty naturally arises from sensor noise, occlusions, incomplete observations, and unexpected state changes. Sequential decision-making under these conditions has been extensively studied within the framework of POMDPs [51]–[54].

A key distinction between a POMDP and an MDP is the introduction of a belief state, which represents a probabilistic

distribution over possible states when the true state cannot be directly observed. Accordingly, a wide range of solution methods have been proposed, including point-based value iteration [55], online tree search [56], and Monte Carlo tree search (MCTS) [57].

Despite these advances, effectively representing and updating the belief state remains a fundamental challenge. Since a belief is inherently a continuous distribution over the state space, exact inference is computationally difficult. Moreover, as the state space grows, the belief space expands combinatorially, making inference increasingly intractable. As a result, various approaches have been proposed to approximate the belief state.

2) *Belief Approximation*: Studies on belief approximation can be broadly categorized into particle-based and factored-based. Particle-based methods approximate the belief as a set of sampled states and have been widely adopted due to their natural compatibility with sampling-based online planning [58], [59]. For example, [57] maintains a set of particles as a history-conditioned belief and estimates action values by sampling future trajectories from a generative model without performing exact Bayesian belief updates. Similar sampling-based ideas have also been extended to large and continuous POMDPs, where DESPOT [51] evaluates a limited set of sampled scenarios and POMCPOW [60] controls search tree growth in continuous action and observation spaces.

In robotic domains, particle-based belief representations are widely used in localization, navigation, SLAM, object search, and manipulation, where particles encode hypotheses over robot, map, and object states under uncertainty. In navigation, [61] combines POMCP with model predictive control to maximize information gain over occupancy-grid maps, while [62] uses hierarchical local online POMDP planning for exploration in large unknown environments. Similar representations have also been used for continuous control under uncertainty, including autonomous driving at occluded intersections [63] and UAV collision avoidance or target tracking under partial observability [64], [65]. In object search and manipulation, particles often represent complete scene hypotheses. For example, [66] samples visible and occluded object configurations for cluttered object search, while [67], [68] uses particle-based scene hypotheses for multi-object manipulation. Collectively, these studies show that particle beliefs provide a flexible approximation for preserving multimodal dependencies among robot, environment, and object states.

In contrast, factored belief representations model the belief by decomposing the state into a set of variables and exploiting their structure [69]–[71]. Such approaches have been widely used in structured probabilistic models, including dynamic Bayesian networks and Hidden Markov Models [72], where the state is represented in a factored form. To improve computational efficiency, various approximate inference methods have been proposed to reduce the complexity of belief updates by relaxing dependencies among variables. For example, [73] approximates belief updates by maintaining a factorized representation while partially accounting for dependencies between variables. While factored representations exploit internal structure of state space, their independence assumptions break in

symbolic planning where facts are tightly linked by action constraints. Therefore, we use a particle-based approximation to preserve these dependencies.

3) *Symbolic Planning under Uncertainty*: Symbolic planning formalisms such as PDDL [15], STRIPS [12], and FDR [18] have been widely used due to their interpretability [74]. These representations are typically based on first-order logic or similar discrete formalisms [75] which describes planning problems through explicit constraints and state transition rules. However, since traditional predicate logic only allows binary true or false values, it is difficult to directly model observation noise or state uncertainty. To address this limitation, various approaches have attempted to integrate probabilistic reasoning into logical representations. For example, [76] assign weights to logical rules to enable probabilistic inference over uncertain relationships, and [77] combine logical structure with probabilistic reasoning.

However, these approaches primarily focus on building inference models, and their high cost in model construction is unsuitable for online decision-making with complex constraints.

As a result, several studies have explored planning under uncertainty while preserving symbolic structure [78], [79]. For example, [80] encodes commonsense knowledge in ASP [75] to construct a prior over possible worlds and integrates it with probabilistic planning [81]. However, since the belief is defined over a set of possible worlds, the computational cost increases as the number of states grows. To address this, some works organize possible worlds in a structured manner. For instance, [82] construct a state space tree and a hierarchy of POMDPs based on domain knowledge, reducing complexity by leveraging abstraction. More recently, large language models have been used for belief approximation, as in [83], which builds a tree of hypotheses and uses LLMs to generate particles in open-ended settings. While these approaches enable more flexible or scalable belief representations, they either depend on predefined hierarchical structures or rely on the quality and consistency of generated hypotheses.

Prior work has explored various belief representations, but global belief approaches remain sample-inefficient under strong symbolic constraints. We instead approximate belief locally by sampling states reachable under each action [81]. These local beliefs are organized in a tree structure, and planning is performed using the off-the-shelf planner POMCP within an interpretable symbolic space. However, uncertainty in real-world robotic systems cannot always be resolved through planning and observation alone. Robots may therefore require additional information to disambiguate conflicting hypotheses, which motivates human corrective systems that selectively request feedback under high uncertainty.

C. Human Corrective System

Despite recent advances in robotic systems, robots still face operational uncertainty in dynamic and unpredictable real-world environments. Prior work has addressed this limitation by involving humans in different forms of robot autonomy [84], [85]. In particular, in human-on-the-loop (HOTL)

systems, robots execute tasks autonomously, while human operators monitor the system and revise or override robot decisions if failures or errors occur [86], [87]. This supervisory structure can improve safety and robustness because human operators can compensate for errors that autonomous systems cannot resolve.

However, HOTL systems require operators to sustain attention and detect failures during robot operation. To provide effective corrective feedback, operators must first notice that the robot is in an uncertain or erroneous state. Situation awareness research suggests that operators must then understand the current situation, project possible consequences, and determine an appropriate intervention [88]. This cognitive process can be difficult in repetitive tasks. Operators may experience vigilance decrement because continuous exposure to monotonous monitoring tasks can slow responses and reduce detection accuracy [89], [90]. Operators may also experience inattention blindness, in which visible failures are missed if attention is not directed to the relevant event [91]. As a result, continuous supervision can increase workload and fatigue [92]. Continuous supervision can also reduce the likelihood that operators detect critical robot failures [92], [93].

Prior work has shown that robots can improve task performance, team efficiency, and subjective satisfaction if they proactively provide assistance during human-robot collaboration [94]. This finding suggests that proactive robot behavior can support collaboration by reducing the need for humans to manage every decision or detect every need for intervention. In parallel, uncertainty-aware robot systems have advanced toward determining when human input is needed. For example, conformal prediction-based methods construct prediction sets with statistical guarantees and use the size of these sets to decide whether to query the user [?], [9], [10]. These methods show that robots can use uncertainty estimates to reduce unnecessary human intervention.

However, studies that develop and evaluate system-initiated query systems have mainly focused on technical performance, such as task performance, uncertainty calibration, and query efficiency. Human factors, such as workload and fatigue, can also affect task performance in automated systems that involve human operators [95]–[97]. Therefore, system-initiated corrective interaction should be evaluated not only in terms of technical performance but also in terms of operator burden and fatigue.

Building on this gap, we examine how system-initiated corrective interaction affect operator workload, fatigue, and task performance in automated robot systems. Our system does not require users to continuously supervise the robot to detect failures. Instead, users provide corrective feedback only if the system requests input. This interaction structure shifts part of the problem detection burden from the user to the robot.

III. PRELIMINARIES

A. Symbolic State and Action Representation

The objective of planning is to find a transition sequence that transforms the initial state into a goal state. The planning

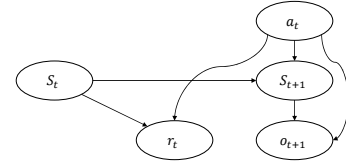


Fig. 2: Graphical illustration of a POMDP. At each time step t , the agent selects an a_t based on its current belief over the latent state s_t . The environment transitions to the next state s_{t+1} and generates a reward r_t . Since the true state is not directly observable, the agent receives an observation o_{t+1} .

problem is defined as a tuple $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, \mathcal{G} \rangle$, where $\mathcal{F}$ is a finite set of **atoms**, $\mathcal{A}$ is a set of actions, $s_0 \subseteq \mathcal{F}$ is the initial state, and $\mathcal{G} \subseteq \mathcal{F}$ is the goal condition. A state s is represented as a subset of $\mathcal{F}$. Each action $a \in \mathcal{A}$ is defined as a tuple $a = \langle \text{pre}(a), \text{add}(a), \text{del}(a) \rangle$, where $\text{pre}(a), \text{add}(a), \text{del}(a) \subseteq \mathcal{F}$ denote the preconditions, add lists, and delete lists of the action, respectively. An action a is said to be **applicable in state s** if and only if $\text{pre}(a) \subseteq s$, denoted as $a \llbracket s \rrbracket$. The state transition is defined as $s \xrightarrow{a} s'$, where $s' = (s \setminus \text{del}(a)) \cup \text{add}(a)$.

A plan is a sequence of actions $\pi = (a_1, \dots, a_T)$ such that the goal condition is satisfied after sequentially applying the actions starting from the initial state, i.e.,

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_T} s_T \quad \text{and} \quad \mathcal{G} \subseteq s_T.$$

B. POMDP

A POMDP is defined as a tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and Ω is the observation space. The transition model $T(s' | s, a)$ specifies the probability of reaching the next state s' after taking action a in current state s . The observation model $O(o | s', a)$ defines the probability of receiving observation o after arriving at state s' by executing action a . The reward function $R(s, a)$ gives a scalar value to each transition, and $\gamma \in (0, 1]$ is the discount factor.

Since the true state is not directly observable, the agent maintains a belief b , which is a probability distribution over $\mathcal{S}$ such that $b(s) = \Pr(s)$. After executing an action a and receiving an observation o , the belief is updated using a Bayes filter.

$$b'(s') = \eta O(o | s', a) \sum_{s \in \mathcal{S}} P(s' | s, a) b(s), \quad (1)$$

where η is a normalization constant.

A policy π maps beliefs to actions. The objective of a POMDP is to find a policy that maximizes the expected cumulative discounted reward, defined as $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$.

C. POMCP

Partially Observable Monte Carlo Planning (POMCP) performs Monte Carlo tree search over action-observation histories $h = \langle a_1 o_1 \dots a_t o_t \rangle$, where each node in the search

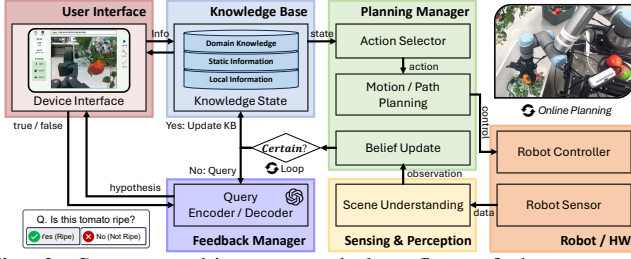


Fig. 3: System architecture and data flow of the proposed framework. The system consists of the *Knowledge Base* (KB), *Planning Manager* (PM), *Sensing and Perception* module (SP), *Feedback Manager* (FM), and *User Interface* (UI). The PM performs online planning based on the current knowledge state stored in the KB and updates the belief using observations from SP. The system evaluates the confidence of the updated belief. If the confidence exceeds a threshold, the most probable hypothesis is committed to the KB. Otherwise, the FM generates a query and requests corrective feedback from the user through the UI. The feedback is then used to refine the belief and continue planning.

tree corresponds to a history. Action selection follows the UCB rule:

$$a^* = \arg \max_a \left(V(ha) + c \sqrt{\frac{\log N(h)}{N(ha)}} \right), \quad (2)$$

where $V(ha)$ is the estimated value of a after h , $N(h)$ and $N(ha)$ denote visit counts, and c is an exploration constant.

Each simulation samples the next state, observation, and reward from a generative model $(s', o, r) \sim \mathcal{G}(s, a)$. The return is computed recursively as $R = r + \gamma R'$, where R' is the return from subsequent steps. The value estimate is updated using an incremental average:

$$V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}. \quad (3)$$

The belief associated with a history h is represented implicitly as a set of particles, $B(h) = \{s \mid s \text{ is visited at } h\}$, which approximates the posterior over states without explicit belief updates.

IV. SYSTEM ARCHITECTURE

In this section, we present an overview of the system architecture and the data flows between modules. Then we give a detailed explanation of their functionalities and applications.

A. Overview

Fig. 3 illustrates the brief system architecture and the data flow between the main modules. We integrate the *Knowledge Base* (KB), the *Planning Manager* (PM), the *Sensing and Perception* module (SP), the *Feedback Manager* (FM), and the *User Interface* (UI). The KB maintains symbolic descriptions and facts used for planning, SP generates sensory information from robot sensors, PM generates and executes plans, FM coordinates corrective feedback, and UI bridges the system and the user.

The KB stores essential information required for the robot to perform tasks and explicitly represents the current state of the system. All modules share information and interact with each other through the KB. The KB is composed of three types of information: domain knowledge, static information, and local information. Domain knowledge refers to prior knowledge in a specific domain. It includes descriptions of robot skills, domain-specific rules, and general explanations of the environment. Static information is used during planning and execution but remains unchanged throughout task execution. Unlike domain knowledge, static information is encoded in a structured form and includes information such as robot parameters and local maps. In contrast, local information is used during planning and execution but cannot be known in advance. It is obtained through observations from the SP module and stored in the KB in the form of facts.

The PM is responsible for online action selection and execution under partial observability. The POMCP-based planner within the PM module selects the best action from the current belief and triggers the SP module to acquire additional observations. For instance, if the robot is positioned in front of a tomato stem, it naturally executes a `detect` action to infer the locations and ripeness states of tomatoes. To quantify uncertainty, the PM approximates the belief in the form of particle and computes the entropy over the resulting belief distribution. If such uncertainty is detected, the PM queries the FM to obtain additional information that can resolve the ambiguity of the observation. Upon receiving a response from the FM, the PM updates the belief accordingly and promotes the most probable belief state to a certain state in the knowledge base.

The SP module provides the current visual sensing information to the UI in real time. In addition, it updates the KB with observations triggered by the PM. In this process, the SP module makes use of data from various sensors, such as cameras, LiDAR, and tactile sensors, to extract information used for task execution, including object locations, states, and properties. For example, from RGBD images, the SP module infers visual attributes such as the position, ripeness, and state of a tomato, while tactile sensors are used to perceive contact-based properties such as object stiffness or softness. The high-level perceptual information extracted by the SP module is triggered by the BT and updated in the KB in the form of symbolic facts.

The FM module is implemented using an LLM-based interface that translates symbolic hypotheses from the PM into user-friendly natural language queries. For example, a hypothesis such as `ripe(tomato1)` is converted into a question like "Is this tomato ripe?". Along with the query, the module provides corresponding visual cues in the user interface, such as highlighting the referenced object with a bounding box. The FM module also interprets user interactions with the interface (e.g., button clicks) and maps them to boolean feedback, which is then incorporated back into the system as validated truth assignments.

Finally, the UI displays the current visual sensing information and the system state, and allows the user to provide direct feedback or make selections. The user interacts with

the FM module through the UI to monitor and supervise task execution.

B. Modules

1) *Knowledge Base*: The KB maintains a structured representation of the environment, including objects, properties, task-relevant relations, numeric quantities, and their associated confidence values. Formally, we define $\mathcal{KB} = \mathcal{I}^D \sqcup \mathcal{I}^S \sqcup \mathcal{I}^L$,¹ where $\mathcal{I}^D$, $\mathcal{I}^S$, and $\mathcal{I}^L$ denote the domain, static, and local information sets, respectively, as described in Sec. IV-A.

For, $\mathcal{I}^S$ and $\mathcal{I}^L$, each information set contains both symbolic facts and numeric fluents. A symbolic fact is defined as $f = \text{pred}(\mathbf{x})$, where $\text{pred}(\cdot)$ denotes a predicate symbol and $\mathbf{x}$ denotes its arguments. A numeric fluent is defined as $\nu = \langle g(\mathbf{x}), v \rangle$, where $g(\mathbf{x})$ denotes a numeric function over objects and $v \in \mathbb{R}$ denotes its value. For example, symbolic facts can represent object categories and relations, such as $\text{tomato}(\text{tomato}_1)$ and $\text{loc}(\text{stem}_1)$, while numeric fluents can represent measurable quantities, such as $\text{pose_x}(\text{tomato}_1) = 0.12$ or $\text{distance}(\text{robot}, \text{stem}_1) = 2.5$ (m).

Specifically, the set $\mathcal{I}^S$ contains information that remains invariant throughout planning. By definition, for all $a \in \mathcal{A}$, $\text{Effect}(a) \cap \mathcal{I}^S = \emptyset$, where $\text{Effect}(a) = \text{add}(a) \cup \text{del}(a)$ denotes symbolic effects and can be extended to include numeric updates. These elements capture structural properties of the environment that are not altered by any action. For example, predicates such as $\text{tomato}(\text{tomato}_1)$ and $\text{loc}(\text{stem}_1)$, as well as fixed numeric quantities like $\text{loc_x}(\text{stem}_1) = 1.2$, remain unchanged regardless of the executed actions.

In contrast, $\mathcal{I}^L$ denotes local information obtained through the SP module and is not known a priori. This includes perceptual predicates, such as object existence or category estimates, as well as numeric fluents estimated from sensor data. For example, predicates such as $\text{ripe}(\text{tomato}_1)$ or $\text{detected}(\text{tomato}_2)$ may be inferred from observations, and numeric values such as $\text{distance}(\text{robot}, \text{stem}_1) = 2.3$ or $\text{pose_x}(\text{tomato}_1) = -0.5$ are updated as new sensor measurements become available.

2) *Planning Manager*: The PM performs decision making based on the current belief state while continuously updating the knowledge base during execution. Uncertainty is quantified using the entropy of the belief distribution. If the uncertainty exceeds a threshold, the planner requests additional verification from the FM. Otherwise, it commits the most probable hypothesis to the knowledge base and proceeds with action execution. In the following, we describe the belief approximation, the uncertainty estimation, the POMCP based search, and the overall planning procedure.

a) *Belief representation*: In partially observable environments, the true state cannot be directly observed, and thus the PM maintains a belief, which is a probability distribution over states. The belief at time t is defined as $b_t(s) = P(s_t = s \mid h_t)$, where $h_t = \{a_1, o_1, \dots, a_t, o_t\}$ denotes the history of actions and observations. However, in symbolic domains, the

state space is large and subject to structural constraints, making exact belief tracking computationally intractable. Thus, we approximate the belief using a set of particles.

Formally, at time step t , the belief is represented as a finite set of particles and their associated weights:

$$\mathcal{B}_t = \{(s_i, w_i)\}_{i=1}^N, \quad \sum_{i=1}^N w_i = 1, \quad (4)$$

where each s_i denotes a possible world composed of a set of facts. The approximated belief distribution is then

$$b_t(s) \approx \sum_{i=1}^N w_i \delta(s_i = s). \quad (5)$$

In the symbolic domain, actions exhibit partially predictable effects due to their interpretable structure. For example, $\text{pick}(\text{tomato}_1)$ may result in grasp failure or selecting a different tomato, but it does not plausibly change the location of the robot. Based on this observation, we define a restricted set of reachable successor states, referred to as the *frontier*. For each s and a , the frontier is defined as

$$\text{Frontier}(s, a) = \{s_1^*, s_2^*, \dots, s_j^*\}, \quad (6)$$

where each s_k^* denotes a reachable successor state.

In contrast to standard particle filtering, we assume a *knowledge* state s^K that represents the set of certain facts. Given a and s^K , we construct the belief by considering only the reachable states induced by s^K . That is, particles are defined exclusively over the frontier $\text{Frontier}(s^K, a)$, and the belief is represented as

$$\mathcal{B}_t = \{(s_k^*, w_k)\}_{k=1}^j, \quad \sum_{k=1}^j w_k = 1, \quad (7)$$

where each s_k^* is a reachable successor state such that $s_k^* \in \text{Frontier}(s^K, a)$. After executing a and receiving an observation o , the belief over the frontier is updated by adjusting the weight of each particle according to how well it explains the observation. Specifically,

$$\forall s_k^* \in \text{Frontier}(s^K, a), \quad w'_k = w_k \cdot O(o \mid s_k^*). \quad (8)$$

followed by normalization to obtain the *uncertainty* of the current state.

b) *Uncertainty of the state*: Given the updated belief over the frontier, uncertainty is quantified based on the distribution of particle weights. Intuitively, uncertainty increases when multiple candidate states have comparable likelihoods, and decreases when the belief is concentrated on a single dominant state. This uncertainty arises from *hypotheses*, which denote literals whose truth values are not yet determined.

The confidence of the current belief is defined as

$$\text{Conf}(\mathcal{B}) = 1 - \frac{\mathcal{H}(\mathcal{B})}{\log_2 j}, \quad (9)$$

where j denotes the number of particles. In the degenerate case where $j = 1$, the belief is fully concentrated on a single state and we define $\text{Conf}(\mathcal{B}) = 1$. Here, $\mathcal{H}(\mathcal{B})$ denotes the entropy of the normalized particle weights. Let $\bar{w}'_k$ denote the

¹The notation $\sqcup$ denotes a disjoint union, indicating that $\mathcal{I}^D$, $\mathcal{I}^S$, and $\mathcal{I}^L$ form a partition of $\mathcal{KB}$.

normalized weights such that $\tilde{w}'_k = w'_k / \sum_{l=1}^j w'_l$. The entropy is computed as

$$\mathcal{H}(\mathcal{B}) = - \sum_{k=1}^j \tilde{w}'_k \log_2 \tilde{w}'_k. \quad (10)$$

This normalization ensures that the confidence lies in $[0, 1]$, where a uniform belief yields low confidence and a concentrated belief yields high confidence.

If the confidence exceeds a predefined threshold τ , the most probable state is selected as the updated knowledge, $s^* = \arg \max_{s_k^*} w'_k$, and the belief collapses to this state by discarding all remaining particles. Otherwise, the system identifies uncertainty-inducing hypotheses that differentiate candidate states.

The candidate hypotheses are constructed from literals that differ between the current knowledge state and the frontier states. Let $\mathcal{C}$ denote this set of candidate hypotheses. For each $c \in \mathcal{C}$, the belief is partitioned into two subsets depending on whether c holds:

$$\mathcal{B}_c^+ = \{(s_k^*, w'_k) \mid c \in s_k^*\}, \quad \mathcal{B}_c^- = \{(s_k^*, w'_k) \mid c \notin s_k^*\}.$$

The system selects the hypothesis that is expected to reduce the uncertainty the most by minimizing the expected entropy after querying:

$$c^* = \arg \min_{c \in \mathcal{C}} [P(c)\mathcal{H}(\mathcal{B}_c^+) + P(\neg c)\mathcal{H}(\mathcal{B}_c^-)], \quad (11)$$

where

$$P(c) = \sum_{s_k^*: c \in s_k^*} w'_k \quad \text{and} \quad P(\neg c) = \sum_{s_k^*: c \notin s_k^*} w'_k.$$

The selected hypothesis c^* is passed to the FM as a query. After receiving the answer, the belief is filtered by removing particles that contradict the feedback. If the answer confirms c^* , only particles containing c^* are retained. Otherwise, particles containing c^* are discarded:

$$\mathcal{B} \leftarrow \begin{cases} \{(s_k^*, w'_k) \mid c^* \in s_k^*\}, & \text{if } ans = \text{true}, \\ \{(s_k^*, w'_k) \mid c^* \notin s_k^*\}, & \text{if } ans = \text{false}. \end{cases} \quad (12)$$

The remaining weights are renormalized, and the confidence is recomputed. This process is repeated until the confidence exceeds τ or no informative hypothesis remains.

c) POMCP-based search: We employ POMCP [57] as the underlying planner for action selection. We use its sampling-based framework and focus on integrating symbolic representations with uncertainty-aware reasoning. Instead, as summarized in Alg. 1 to Alg. 3, we apply minimal adaptations to ensure consistency with the symbolic domain.

In Alg. 1, the search phase follows standard POMCP by repeatedly sampling a state and performing simulations. Each sampled state is passed to `SIMULATE` (line 11), and the accumulated values are used to select the final action. If the belief is empty, the search uses s^K , and at the root, the action is chosen based on value without UCB (line 13).

In the simulation phase, as described in Alg. 2, the recursive structure and return computation remain the same as in standard POMCP, but both action selection and node

Algorithm 1: POMCP Search with Applicable Actions

```

1 Input: belief  $b$ 
2 Output: selected action  $a \in \mathcal{A}$ 
3 procedure SEARCH( $b$ )
4    $h \leftarrow$  root history
5    $s^K \leftarrow knowledge(b)$ 
6   repeat
7     if  $B(h) \neq \emptyset$  then
8       | sample  $s \sim B(h)$ 
9     else
10      |  $s \leftarrow s^K$ 
11      $SIMULATE(s, h, 0)$ 
12   until TIMEOUT;
13    $a \leftarrow SEARCH-BEST(h, s^K, false)$ 
14   return  $a$ 
```

expansion are restricted to $\mathcal{A}(s)$ (line 10). That is, instead of considering the full action space, only actions applicable in state s are explored. If a new history h is encountered, the node is expanded only with actions in $\mathcal{A}(s)$ (line 11), and the value is initialized via a ROLLOUT. Here, T denotes the search tree, while $N(ha)$ and $V(ha)$ represent the visit count and value of action a , respectively. The next state, observation, and reward are then sampled from the generative model $(s', o, r) \sim \mathcal{G}(s, a)$ (line 15). As a result, the simulation remains stochastic but is confined to reachable symbolic transitions.

In Alg. 3, the same principle is applied to both action selection and rollout. The candidate set $\mathcal{C} = \{a \mid a \llbracket s \rrbracket \in \mathcal{A}, ha \in T\}$ includes actions that are both applicable in state s and present in the tree. If `use_ucb` is true, an action is selected according to the UCB criterion defined in Eq. 2. Otherwise, the action with the highest value is chosen. During rollout, actions are sampled uniformly from all applicable actions. The generative model $\mathcal{G}(s, a)$ then produces the next state, observation, and reward, while the action space is restricted by symbolic constraints.

In symbolic domains, action preconditions and effects are explicitly defined, so exploring infeasible actions is unnecessary. Restricting the action space to $\mathcal{A}(s)$ therefore improves both search efficiency and model consistency. Overall, the structure follows POMCP while naturally incorporating the constraints imposed by the symbolic representation.

d) Action execution: Each $a \in \mathcal{A}$ selected by the POMCP is associated with an executable robot skill. The symbolic action specifies the task-level intention and parameters, while the corresponding skill defines the motion or perception routine required for execution. For example, a symbolic action such as `navigate(robot,  $l_1, l_2$ )` is mapped to a navigation routine, and `pick(robot,  $tomato_1$ )` is mapped to a predefined manipulation trajectory.

In this work, $\forall a \in \mathcal{A}$ follows an encoded execution policy that maps a symbolic operator to a robot skill through an off-the-shelf motion planner, such as MoveIt [98]. The planner takes symbolic facts and numeric fluents stored in the KB as

Algorithm 2: POMCP Simulation with Applicable Actions

```

1 Input: state  $s$ , history  $h$ , depth  $depth$ 
2 Output: simulated return  $R$ 
3 procedure SIMULATE ( $s, h, depth$ )
4   if  $\gamma^{depth} < \epsilon$  then
5     return 0
6   if there is no  $a[s] \in \mathcal{A}$  then
7      $N(h) \leftarrow N(h) + 1$ 
8     return 0
9   if  $h \notin T$  then
10    foreach  $a[s] \in \mathcal{A}$  do
11       $T, N(ha), V(ha) \leftarrow T \cup \{ha\}, 0, 0$ 
12       $N(h) \leftarrow N(h) + 1$ 
13      return ROLLOUT ( $s, depth$ )
14    $a \leftarrow \text{SEARCH-BEST}(h, s, \text{true})$ 
15   sample  $(s', o, r)$  from simulator  $\mathcal{G}(s, a)$ 
16    $R \leftarrow r + \gamma \cdot \text{SIMULATE}(s', hao, depth + 1)$ 
17    $B(h) \leftarrow B(h) \cup \{s\}$ 
18    $N(h) \leftarrow N(h) + 1$ 
19    $N(ha) \leftarrow N(ha) + 1$ 
20    $V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}$ 
21   return  $R$ 

```

Algorithm 3: Action Selection and Rollout

```

1 Input: history  $h$ , state  $s$ 
2 procedure SEARCH-BEST ( $h, s, use\_ucb$ )
3    $\mathcal{C} \leftarrow \{a \mid a[s] \in \mathcal{A}, ha \in T\}$ 
4   if  $\mathcal{C} = \emptyset$  then
5     return  $\emptyset$ 
6   if  $use\_ucb$  then
7     return  $\arg \max_{a \in \mathcal{C}} V(ha) + c \cdot \sqrt{\frac{\log N(h)}{1 + N(ha)}}$ 
8   else
9     return  $\arg \max_{a \in \mathcal{C}} V(ha)$ 
10 procedure ROLLOUT ( $s, depth$ )
11   if  $(\gamma^{depth} < \epsilon \vee \gamma = 0) \wedge depth > 0$  then
12     return 0
13   if there is no  $a[s] \in \mathcal{A}$  then
14     return 0
15    $a \sim \text{Uniform}(\{a \mid a[s] \in \mathcal{A}\})$ 
16   sample  $(s', o, r)$  from simulator  $\mathcal{G}(s, a)$ 
17   return  $r + \gamma \cdot \text{ROLLOUT}(s', depth + 1)$ 

```

inputs. Numeric fluents, such as object poses, distances, and grasping parameters, define the parameters of motion generation and maintain consistency between high-level planning and low-level execution.

Specifically, after each action execution, the system receives an observation o from the environment and uses it for belief

Algorithm 4: Planning Pipeline

```

1 Input: initial knowledge  $s_0^K$ , confidence threshold  $\tau$ 
2 Output: plan  $\pi$ 
3 procedure PLANNING ( $s_0^K$ )
4    $knowledge(\mathcal{B}) \leftarrow s_0^K$ 
5    $t \leftarrow 0$ 
6   repeat
7      $a_t \leftarrow \text{SEARCH}(\mathcal{B})$ 
8     if  $a_t = \emptyset$  then
9       // There is no  $a[s]$ . (Dead-end)
10      break
11     execute  $a_t$  and receive observation  $o_{t+1}$ 
12      $\mathcal{B} \leftarrow \text{UPDATE}(\mathcal{B}, a_t, o_{t+1})$  (Eq. 8)
13      $c \leftarrow \text{CONF}(\mathcal{B})$  (Eq. 9)
14     while  $c < \tau$  do
15        $hypo^* \leftarrow \text{SELECT-HYPO}(\mathcal{B})$  (Eq. 11)
16        $ans \leftarrow \text{QUERY-FM}(hypo^*)$ 
17        $\mathcal{B} \leftarrow \text{FILTER}(\mathcal{B}, hypo^*, ans)$  (Eq. 12)
18        $c \leftarrow \text{CONF}(\mathcal{B})$ 
19      $s^K \leftarrow \arg \max_{s_k^* \in \mathcal{B}} w'_k$ 
20      $knowledge(\mathcal{B}) \leftarrow s^K$ 
21      $t \leftarrow t + 1$ 
22   until  $t = \text{MAX-STEP}$ ;
23   return  $\pi = (a_0, \dots)$ 

```

update. The system uses these observations to update both symbolic predicates and numeric fluents in the KB. This design separates task-level decision making from low-level motion planning. The KB provides a shared representation across these levels. As a result, the PM does not depend on a specific motion planner implementation.

e) *Overall planning pipeline:* We adopt a online planning framework as described in Algorithm 4. The planning process starts from the initial knowledge state s_0^K , which is used to initialize the belief (line 4). At each iteration, the planner first selects a_t by performing Monte Carlo tree search over the current belief $\mathcal{B}$ (line 7). If no applicable action exists, the process terminates. Otherwise, the selected action is executed and an observation o_{t+1} is received from the environment.

The belief is then updated using the observation model (line 11), and the confidence of the updated belief is computed (line 12). If the confidence is below the threshold τ , the planner queries the FM to reduce uncertainty. Specifically, it selects the most informative hypothesis, sends it to the FM, and filters out particles inconsistent with the answer (lines 14–16). This process is repeated until the confidence exceeds τ . Once the belief becomes sufficiently confident, the most probable state is committed as the updated knowledge state, and the planner proceeds to the next step.

3) *Sensing and Perception:* The SP module acquires raw sensory data, generates local information in $\mathcal{I}^L$, and updates the numeric fluents in the KB. It also provides observations required for belief updates during action execution. Let $\mathcal{M}$ denote the set of perception models. For each predicate

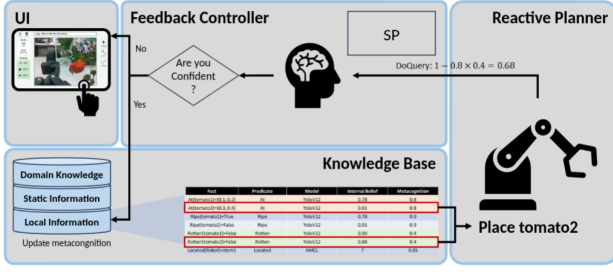


Fig. 4: An overview of FM system. This figure is a temporary image.

pred associated with local information in $\mathcal{I}^L$, there exists a corresponding perception model $m_{\text{pred}} \in \mathcal{M}$ defined as

$$m_{\text{pred}} : \mathcal{R} \rightarrow \mathcal{I}^L,$$

where $\mathcal{R}$ denotes the space of raw sensory data. Given a sensory input $r \in \mathcal{R}$, the model $m_{\text{pred}}(r)$ produces a symbolic fact $f = \text{pred}(\mathbf{x})$ or a numeric fluent $\nu = \langle g(\mathbf{x}), v \rangle$ in $\mathcal{I}^L$.

Through this process, perceptual predicates such as $\text{ripe}(\text{tomato}_1)$ or $\text{detected}(\text{tomato}_2)$ are instantiated, and numeric fluents such as $\nu = \langle \text{distance}(\text{robot}, \text{stem}_1), 2.3 \rangle$ or $\nu = \langle \text{pose_x}(\text{tomato}_1), -0.5 \rangle$ are continuously updated based on incoming observations. Since $\mathcal{I}^L$ represents information that is not known a priori, all elements in this set are generated and refined exclusively through the SP module.

In addition, the SP module defines the observation interface of the system. After executing an action $a \in \mathcal{A}$, the system receives an observation $o \in \mathcal{O}$ derived from the current sensory input, which is used to update the belief over possible states. Furthermore, if triggered by the PM, the SP module acquires new sensory input and evaluates the relevant perception models to produce observations and update the associated elements in $\mathcal{I}^L$.

4) *Feedback Manager*: As illustrated in Fig. 4, the FM translates hypotheses provided by the PM into a form that is easily understandable for the user, and interprets user responses into a format that can be processed by the system. Specifically, a hypothesis such as $\text{ripe}(\text{tomato}_1)$ is represented internally as a predicate, but the FM converts it into a natural question such as “Is this tomato ripe?”. To support user understanding, the queried object is visualized with a bounding box indicating its location.

In addition, the FM interprets user inputs. Users can respond via touch or speech, and the FM converts these responses into a boolean form (true or false) that can be used for belief updates. Through this bidirectional translation, the FM reduces the semantic gap between the system and the user, enabling efficient interaction.

5) *User Interface*: As shown in Fig. 5, the UI enables interaction between the user and the system and is implemented on a tablet. It presents queries generated by the FM and overlays the queried object on the camera image using a bounding box to support intuitive understanding. Users provide responses through simple touch input, with optional speech input. The interface hides internal representations and is designed to elicit accurate feedback with minimal workload.



Fig. 5: An overview of interface. This figure is a temporary image.



(a) Waste sorting domain overview.



(b) Tomato harvesting domain overview

Fig. 6: The illustration of the environments. (a) A UR5 robot sorts four types of waste into designated bins in Isaac Sim. (b) A mobile manipulator harvests ripe tomatoes and discards rotten ones in a real-world setting.

V. EXPERIMENTS

We evaluate the proposed system through two system-level experiments. First, we analyze how the confidence threshold τ affects task success, execution length, and query frequency. This experiment characterizes the trade-off between autonomous execution policies and human intervention. Second, we compare four query policies: **All**, **No**, **Random**, and **Ours**. If a query is triggered, we assume that the human provides the correct answer, which isolates the effect of the query policy from user response errors. A separate user study is then used to examine the user-facing effects of these interaction policies.

A. Experimental Setup

We evaluate the proposed method in two domains: a simulated waste sorting domain and a real-world tomato harvesting domain. The waste sorting domain is implemented in simulation using NVIDIA Isaac Sim, where a fixed UR5 robotic arm is used for pick and place task. Object detection is performed using the YOLOv12 model [99]. In contrast,

the tomato harvesting domain is deployed on a real mobile manipulator consisting of a UR5e robotic arm mounted on a Summit mobile platform. The robot is equipped with an RGB-D camera for visual perception and a LiDAR sensor for localization, and performs object detection using the same YOLOv12.

1) *Waste sorting domain:* We consider a tabletop waste-sorting scenario with four waste objects placed on a table, as shown in Fig. 6a. A fixed UR5 robot is positioned between the objects and the recycling bins and performs three actions: detect, pick, and place. We assume that all objects are reachable by the manipulator.

Each object belongs to one of four categories: *general*, *paper*, *can*, and *plastic*. The object categories are not known a priori and must be inferred through perception. Since objects may be partially occluded by others, a single observation may not reveal all categories, requiring repeated detection during execution. The *detect* action can observe multiple objects simultaneously, after which the robot places each object into the corresponding recycling bin.

The initial arrangement of objects is randomized in every episode. A task is considered successful when all objects are correctly sorted within the maximum step limit. A reward of +5 is assigned when a waste object is placed into the recycling bin corresponding to its category, and an additional reward of +10 is given when all objects are correctly sorted. All other actions receive zero reward.

2) *Tomato harvesting domain:* We consider a tomato harvesting scenario in which four tomatoes are attached to two stems, denoted as *stem_1* and *stem_2*, as shown in Fig. 6b. The robot navigates between three locations: *dock_station*, *stem_1*, and *stem_2*, and performs detect, pick, scan, place, and discard actions.

Each tomato can be in one of three states: *ripe*, *unripe*, or *rotten*. The tomato states are initially unknown and must be estimated through perception. During detection, ripe and rotten tomatoes are visually ambiguous and cannot always be reliably distinguished. To resolve this ambiguity, the robot performs a *scan* action after picking a tomato to verify its condition before harvesting or discarding it. Unripe tomatoes are left untouched.

The tomato locations are randomized across stems in every episode. A task is considered successful when all ripe tomatoes are harvested and all rotten tomatoes are discarded within the maximum step limit. A reward of +5 is assigned when a ripe tomato is correctly harvested or a rotten tomato is correctly discarded after scanning. An additional reward of +10 is given when all task-relevant tomatoes are processed correctly. All other actions receive zero reward.

3) *Implementation Details:* In all experiments, the planner used a discount factor $\gamma = 0.95$, UCB exploration constant $c = 1.0$, maximum simulation depth $depth_{\max} = 20$, and $TIMOUT = 100$ Monte Carlo simulations per planning step. Each episode was limited to 50 steps. For the policy comparison experiment, the confidence threshold for triggering user feedback was set to $\tau = 0.8$ based on the threshold sweep reported in Sec. V-B.

B. System Evaluation

1) Experimental Results:

We first evaluate the effect of the confidence threshold τ on system performance. Fig. ?? and Table I report the average number of queries, execution steps, and task success rate under different threshold values.

As τ increases, the average number of queries generally increases in both domains. In waste sorting, increasing τ from 0.6 to 0.7 changes the success rate from 58.0% to 92.5%, the average number of queries from 1.8 to 3.2, and the average number of execution steps from 24.2 to 10.6. In tomato harvesting, the same change results in a success rate increase from 28.0% to 95.5%, a query count increase from 12.7 to 22.2, and a reduction in execution steps from 40.1 to 19.7. For $\tau \geq 0.7$, the success rate remains above 92% in both domains. At $\tau = 0.8$, the system achieves success rates of 95.5% and 98.0% in waste sorting and tomato harvesting, respectively. The corresponding average numbers of queries are 8.1 and 24.3, and the average numbers of execution steps are 10.6 and 20.2. Based on these results, we use $\tau = 0.8$ in the remaining experiments.

We next compare four query policies at $\tau = 0.8$. **All** requests feedback whenever a query can be generated. **No** performs fully autonomous execution without user feedback. **Random** requests feedback randomly. **Ours** requests feedback when the confidence of the current belief falls below the threshold and selects the query with the highest expected information gain.

Fig. ?? and Table II summarize the results. In waste sorting, **All**, **Random**, and **Ours** achieve success rates of 100.0%, whereas **No** achieves 80.0%. The average numbers of queries are 15.8, 6.8, and 9.2 for **All**, **Random**, and **Ours**, respectively. In tomato harvesting, **Ours** achieves a success rate of 98.0%, compared to 96.5% for **All**, 20.0% for **No**, and 1.0% for **Random**. The corresponding average numbers of queries are 20.5, 37.3, 0.0, and 16.7. The average numbers of execution steps are 20.2, 22.4, 36.9, and 42.3, respectively.

2) Discussion:

The results demonstrate that human feedback can improve both task performance and execution efficiency when it is requested selectively. As the confidence threshold increases, the robot acquires additional observations through user feedback, leading to substantial improvements in task success rate. At the same time, the average execution length decreases in both domains. For example, increasing τ from 0.6 to 0.7 improves the success rate from 58.0% to 92.5% in waste sorting and from 28.0% to 95.5% in tomato harvesting, while reducing the average execution length by more than half in both domains.

This result suggests that informative observations can guide the planner toward more effective action sequences. If uncertainty remains unresolved, the robot may act on incorrect beliefs, resulting in unnecessary actions or failed execution attempts. By obtaining feedback at uncertain states, the planner can refine its belief earlier and select actions that are more consistent with the underlying state of the environment. Consequently, additional observations can reduce the total number of actions required to complete the task.

TABLE I: System evaluation under different confidence thresholds τ . Success rates are reported as percentages.

τ	Waste Sorting			Tomato Harvesting		
	Success Rate (%)	Avg. Queries	Avg. Steps	Success Rate (%)	Avg. Queries	Avg. Steps
0.0	62.5	0.0	9.7	0.0	0.0	5.8
0.1	54.5	0.6	8.4	0.0	0.1	5.8
0.2	47.5	0.8	28.5	21.0	3.1	39.6
0.3	51.5	0.8	28.4	18.5	3.3	40.7
0.4	50.5	0.8	26.9	8.5	4.4	46.2
0.5	56.5	1.4	24.9	15.5	10.3	43.6
0.6	58.0	1.8	24.2	28.0	12.7	40.1
0.7	92.5	3.2	10.6	95.5	22.2	19.7
0.8	95.5	8.1	10.6	98.0	24.3	20.2
0.9	96.0	8.8	11.2	97.5	24.8	20.8
1.0	93.1	15.3	10.6	97.5	43.5	23.5

TABLE II: System evaluation under different querying strategies at $\tau = 0.8$. **All** queries whenever feedback is available, **No** performs fully autonomous execution, **Random** queries without using belief confidence, and **Ours** selectively queries based on belief confidence.

Method	Waste Sorting			Tomato Harvesting		
	Success Rate (%)	Avg. Queries	Avg. Steps	Success Rate (%)	Avg. Queries	Avg. Steps
All	100.0	15.8	11.2	96.5	37.3	22.4
No	80.0	0.0	10.8	20.0	0.0	36.9
Random	100.0	6.8	10.4	1.0	16.7	42.3
Ours	100.0	9.2	11.2	98.0	20.5	20.2

The comparison among query policies further highlights the importance of selecting when to request feedback. Although the **All** policy achieves high success rates, it requires substantially more user interactions. In tomato harvesting, **All** requires 37.3 queries on average, whereas **Ours** requires only 20.5 while achieving a slightly higher success rate. A similar trend is observed in waste sorting, where both methods achieve perfect success rates but **Ours** reduces the number of queries from 15.8 to 9.2. These results indicate that many feedback requests generated by continuous supervision provide limited additional information.

In contrast, the **No** policy often fails because belief errors accumulate without correction, while the **Random** policy demonstrates that simply increasing the amount of feedback is insufficient. Feedback must be acquired at states where uncertainty is likely to affect future decisions. By focusing interactions on low-confidence beliefs, the proposed approach maintains high task success, reduces unnecessary user involvement, and shortens execution length.

C. User Study

We conducted a Wizard-of-Oz (WoZ) user study to evaluate how different query policies affect user workload, situation awareness, and cognitive fatigue during human-robot collaboration. Unlike the system evaluation in Section V-B, which focused on autonomous task performance, this study focused on the user experience associated with different interaction strategies. Robot behavior was controlled using predefined scenario scripts, while participants interacted with the system

through the same tablet-based interface used by the feedback manager.

1) Participants:

Twelve participants (7 female and 5 male) took part in the study. Participants ranged in age from 21 to 26 years ($M = 23.0$, $SD = 1.48$). Each participant received \$20 upon completion of the study.

2) Study Design and Conditions:

The study employed a within-subject design. Each participant completed ten trials in total, consisting of five tomato harvesting scenarios and five waste sorting scenarios.

In each domain, participants experienced five interaction conditions: **All**, **No**, **Ours1**, **Ours2**, and **KnowNo**. **All** requested feedback whenever a query was available, representing continuous human supervision. **No** performed the task without requesting user feedback. **Ours1** used the proposed uncertainty-based state query policy. **Ours2** used a variant of the proposed policy designed to expose participants to a broader range of query situations. **KnowNo** served as an action-level query baseline.

Each domain consisted of five scenarios and five interaction conditions. The assignment between scenarios and conditions was randomized for each participant such that every participant experienced all five conditions exactly once per domain. This design prevented a particular condition from being consistently associated with a specific scenario. Detailed scenario descriptions are provided in Appendix ??.

3) Materials:

Participants interacted with the robot through a tablet-based interface. The interface displayed the current task state, high-

lighted task-relevant objects, and presented robot-generated questions together with selectable answer options.

The tomato harvesting scenarios required participants to identify the state of tomatoes, such as whether a tomato was ripe, unripe, or rotten. The waste sorting scenarios required participants to identify the category of waste objects, including general waste, paper, cans, and plastic. Example interface screenshots are shown in Fig. ??.

To ensure a consistent interaction sequence across participants, we adopted a WoZ protocol. The experimenter advanced the robot state according to predefined scenario scripts and recorded participant responses. If a participant selected an incorrect answer, the response was recorded as incorrect for evaluation purposes, but the scenario continued using the intended task state so that subsequent interactions were not affected by earlier mistakes.

4) Procedure:

Before the experiment, participants received an explanation of the study objectives, the two task domains, and the robot interaction process. They were then introduced to the tablet interface and completed several practice interactions to become familiar with the available information and response procedure.

During the main session, participants completed ten trials without assistance. In each trial, participants observed the robot execution through the interface and responded whenever the system requested feedback. After each scenario, participants completed a questionnaire regarding the interaction experience. After completing all ten trials, participants completed post-study questionnaires and provided qualitative feedback during a short debriefing session.

5) Measures:

We collected both subjective and objective measures.

Subjective workload was measured using NASA Raw-TLX. Situation awareness was evaluated using SAGAT-style questionnaires and SART. Cognitive fatigue was measured using a self-reported fatigue questionnaire administered after each task condition. Unless otherwise specified, questionnaire responses were recorded using a 5-point Likert scale.

For SAGAT, each scenario included six multiple-choice questions concerning the current task state, the source of robot uncertainty, and the expected consequences of user inaction. Responses were scored as correct or incorrect using predefined answer keys, and the resulting accuracy was used as a measure of situation awareness.

In addition, we recorded behavioral measures including task completion time, questionnaire completion time, and participant responses for each condition and scenario.

VI. CONCLUSION AND FUTURE WORK

In this paper, we proposed a robotic decision-making framework that integrates symbolic knowledge and uncertainty, with the goal of reducing user workload. The proposed method maintains a belief over possible worlds within a POMDP-based planning framework and quantitatively models uncertainty through a probability distribution over these worlds. Based on this, the robot can assess its own uncertainty and selectively query the human only when necessary. This approach

enables stable task execution without continuous supervision and minimizes unnecessary interactions, thereby reducing user intervention.

In particular, we leverage entropy-based confidence to determine when to issue queries and select the most informative hypothesis to construct questions, improving the efficiency of interaction. Experimental results show that the proposed method reduces the number of human interactions while maintaining or improving task success rates, and effectively lowers user workload. User studies further support this trend, demonstrating that a system which actively manages uncertainty and requests assistance only when needed can operate effectively in real-world settings.

For future work, several extensions can be considered. First, since the current query mechanism is based on symbolic hypotheses, it can be extended to a more intuitive natural language interface to improve user experience. In addition, incorporating more refined models of perception uncertainty would improve the accuracy of uncertainty estimation. Furthermore, this approach can be extended to multi-robot settings, where multiple agents share uncertainty and coordinate their queries collaboratively.

ACKNOWLEDGEMENT

This research was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT)(No. RS-2024-00461583) and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00411007).

REFERENCES

- [1] K. N. Kaipa, A. S. Kankanahalli-Nagendra, N. B. Kumbla, S. Shriyam, S. S. Thevendria-Karthic, J. A. Marvel, and S. K. Gupta, "Addressing perception uncertainty induced failure modes in robotic bin-picking," *Robotics and Computer-Integrated Manufacturing*, vol. 42, pp. 17–38, 2016.
- [2] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.
- [3] H. Kurniawati and V. Yadav, "An online pomdp solver for uncertainty planning in dynamic environment," in *Robotics Research: The 16th International Symposium ISRR*. Springer, 2016, pp. 611–629.
- [4] A. D. Dragan, K. C. Lee, and S. S. Srinivasa, "Legibility and predictability of robot motion," in *International Conference on Human-Robot Interaction (HRI)*. IEEE, 2013, pp. 301–308.
- [5] M. A. Goodrich and A. C. Schultz, "Human-robot interaction: a survey," *Foundations and trends® in human-computer interaction*, vol. 1, no. 3, pp. 203–275, 2008.
- [6] J. Y. Chen and M. J. Barnes, "Human-agent teaming for multirobot control: A review of human factors issues," *IEEE Transactions on Human-Machine Systems*, vol. 44, no. 1, pp. 13–29, 2014.
- [7] D. Sadigh, A. D. Dragan, S. S. Sastry, and S. A. Seshia, "Active preference-based learning of reward functions," in *Proceedings of Robotics: Science and Systems (RSS)*, 2017. [Online]. Available: <https://escholarship.org/uc/item/88k894w7>
- [8] J. Leusmann, C. Wang, M. Gienger, A. Schmidt, and S. Mayer, "Understanding the uncertainty loop of human-robot interaction," *arXiv preprint arXiv:2303.07889*, 2023.
- [9] K. Liang, Z. Zhang, and J. F. Fisac, "Introspective planning: Aligning robots' uncertainty with inherent task ambiguity," *Advances in Neural Information Processing Systems*, vol. 37, pp. 71 998–72 031, 2024.
- [10] J. F. Mullen and D. Manocha, "Lbap: Improved uncertainty alignment of llm planners using bayesian inference," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 18 716–18 723.

- [11] R. Davis, H. Shrobe, and P. Szolovits, "What is a knowledge representation?" *AI magazine*, vol. 14, no. 1, pp. 17–17, 1993.
- [12] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.
- [13] R. Brachman and H. Levesque, *Knowledge representation and reasoning*. Elsevier, 2004.
- [14] S. Russell, P. Norvig, and A. Intelligence, "A modern approach," *Artificial Intelligence*. Prentice-Hall, Englewood Cliffs, vol. 25, no. 27, pp. 79–80, 1995.
- [15] C. Aeronautiques, A. Howe, C. Knoblock, I. D. McDermott, A. Ram, M. Veloso, D. Weld, D. W. Sri, A. Barrett, D. Christianson *et al.*, "PDDL—the planning domain definition language," *Technical Report, Tech. Rep.*, 1998.
- [16] J. Johnson, R. Krishna, M. Stark, L.-J. Li, D. Shamma, M. Bernstein, and L. Fei-Fei, "Image retrieval using scene graphs," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3668–3678.
- [17] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, "3d scene graph: A structure for unified semantics, 3d space, and camera," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 5664–5673.
- [18] M. Helmert, "Concise finite-domain representations for pddl planning tasks," *Artificial Intelligence*, vol. 173, no. 5-6, pp. 503–535, 2009.
- [19] J. Hoffmann, "Ff: The fast-forward planning system," *AI magazine*, vol. 22, no. 3, pp. 57–57, 2001.
- [20] M. Helmert, "The fast downward planning system," *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.
- [21] A. B. Corrêa and G. De Giacomo, "Lifted planning: Recent advances in planning using first-order representations," in *IJCAI*, 2024, pp. 8010–8019.
- [22] M. Tenorth and M. Beetz, "Knowledge processing for autonomous robot control," in *Proc. of AAAI Spring Symposium: Designing Intelligent Robots*, 2012.
- [23] Z. Liu, M. Jiang, and H. Lin, "A graph-based spatial temporal logic for knowledge representation and automated reasoning in cognitive robots," 2020.
- [24] L. Kunze, K. K. Doreswamy, and N. Hawes, "Using qualitative spatial relations for indirect object search," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 163–168.
- [25] S. Hasegawa, A. Taniguchi, Y. Hagiwara, L. El Hafi, and T. Taniguchi, "Inferring place-object relationships by integrating probabilistic logic and multimodal spatial concepts," in *Proc. of IEEE/SICE International Symposium on System Integration (SII)*, 2023, pp. 1–8.
- [26] T. Mota, M. Sridharan, and A. Leonardis, "Integrated commonsense reasoning and deep learning for transparent decision making in robotics," *SN Computer Science*, vol. 2, no. 4, p. 242, 2021.
- [27] S. Zhang, P. Khandelwal, and P. Stone, "iCORPP: Interleaved commonsense reasoning and probabilistic planning on robots," *Robotics and Autonomous Systems*, vol. 174, p. 104613, 2024.
- [28] A. Daruna, W. Liu, Z. Kira, and S. Chetnova, "RoboCSE: Robot common sense embedding," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 9777–9783.
- [29] A. Mitrevsk, P. G. Plöger, and G. Lakemeyer, "Ontology-assisted generalisation of robot action execution knowledge," in *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 6763–6770.
- [30] A. Olivares-Alarcos, D. Beßler, A. Khamis, P. Goncalves, M. K. Habib, J. Bermejo-Alonso, M. Barreto, M. Diab, J. Rosell, J. Quintas *et al.*, "A review and comparison of ontology-based approaches to robot autonomy," *The Knowledge Engineering Review*, vol. 34, p. e29, 2019.
- [31] S. Manzoor, Y. G. Rocha, S.-H. Joo, S.-H. Bae, E.-J. Kim, K.-J. Joo, and T.-Y. Kuc, "Ontology-based knowledge representation in robotic systems: A survey oriented toward applications," *Applied Sciences*, vol. 11, no. 10, p. 4324, 2021.
- [32] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," 2022.
- [33] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar *et al.*, "Inner monologue: Embodied reasoning through planning with language models," 2022.
- [34] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "ProgPrompt: Program generation for situated robot task planning using large language models," *Autonomous Robots*, vol. 47, no. 8, pp. 999–1012, 2023.
- [35] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500.
- [36] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, "Task and motion planning with large language models for object rearrangement," in *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 2086–2092.
- [37] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa *et al.*, "Concept-graphs: Open-vocabulary 3d scene graphs for perception and planning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 5021–5028.
- [38] A. Leanza, A. Moroncelli, G. Vizzari, F. Braghin, L. Roveda, and B. Spahiu, "Conceptbot: Enhancing robot's autonomy through task decomposition with large language models and knowledge graph," *arXiv preprint arXiv:2509.00570*, 2025.
- [39] S. Singh, K. Swaminathan, N. Dash, R. Singh, S. Banerjee, M. Sridharan, and M. Krishna, "Adaptbot: Combining llm with knowledge graphs and human input for generic-to-specific task decomposition and knowledge refinement," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4345–4351.
- [40] J. Zhou, J. Miao, X. Wang, and J. Yu, "Enhancing llm planning for robotics manipulation through hierarchical procedural knowledge graphs," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [41] W. Xu, M. Wang, W. Zhou, and H. Li, "P-RAG: Progressive retrieval augmented generation for planning on embodied everyday task," in *Proc. of ACM International Conference on Multimedia (MM)*, 2024, pp. 6969–6978.
- [42] Z. Wang, S. X. Teo, J. J. Chew, and W. Shi, "InstructRAG: Leveraging retrieval-augmented generation on instruction graphs for llm-based task planning," in *Proc. of International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2025.
- [43] N. Kumar, W. Shen, F. Ramos, D. Fox, T. Lozano-Pérez, L. P. Kaelbling, and C. R. Garrett, "Open-world task and motion planning via vision-language model generated constraints," *IEEE Robotics and Automation Letters*, 2026.
- [44] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3d value maps for robotic manipulation with language models," in *Proc. of Conference on Robot Learning (CoRL)*, 2023, pp. 540–562.
- [45] Z. Dai, A. Asgharivaskasi, T. Duong, S. Lin, M.-E. Tzes, G. Pappas, and N. Atanasov, "Optimal scene graph planning with large language model guidance," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 14 062–14 069.
- [46] Y. Liu, L. Palmieri, S. Koch, I. Georgievski, and M. Aiello, "DELTA: Decomposed efficient long-term robot task planning using large language models," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 10 995–11 001.
- [47] D. Paulius, A. Agostini, B. Quartey, and G. Konidaris, "Bootstrapping object-level planning with large language models," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 16 233–16 239.
- [48] B. Quartey, E. Rosen, S. Tellex, and G. Konidaris, "Verifiably following complex robot instructions with foundation models," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 1–8.
- [49] F. Yang, D. Lyu, B. Liu, and S. Gustafson, "PEORL: Integrating symbolic planning and hierarchical reinforcement learning for robust decision-making," 2018.
- [50] A. Bouguerra, L. Karlsson, and A. Saffiotti, "Monitoring the execution of robot plans using semantic knowledge," *Robotics and Autonomous Systems*, vol. 56, no. 11, pp. 942–954, 2008.
- [51] A. Somani, N. Ye, D. Hsu, and W. S. Lee, "Despot: Online pomdp planning with regularization," *Advances in neural information processing systems*, vol. 26, 2013.
- [52] H. Bai, S. Cai, N. Ye, D. Hsu, and W. S. Lee, "Intention-aware online pomdp planning for autonomous driving in a crowd," in *2015 IEEE international conference on robotics and automation (icra)*. IEEE, 2015, pp. 454–460.
- [53] M. Cramer, K. Kellens, and E. Demeester, "Probabilistic decision model for adaptive task planning in human-robot collaborative assembly based on designer and operator intents," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7325–7332, 2021.
- [54] R. Z. B. Bravo, A. Leiras, and F. L. Cyrino Oliveira, "The use of UAVs in humanitarian relief: An application of pomdp-based methodology for

- finding victims,” *Production and Operations Management*, vol. 28, no. 2, pp. 421–440, 2019.
- [55] J. Pineau, G. Gordon, S. Thrun *et al.*, “Point-based value iteration: An anytime algorithm for pomdps,” in *Ijcai*, vol. 3, 2003, pp. 1025–1032.
- [56] S. Ross, J. Pineau, S. Paquet, and B. Chaib-Draa, “Online planning algorithms for pomdps,” *Journal of Artificial Intelligence Research*, vol. 32, pp. 663–704, 2008.
- [57] D. Silver and J. Veness, “Monte-carlo planning in large pomdps,” *Advances in neural information processing systems*, vol. 23, 2010.
- [58] P. Doshi and P. J. Gmytrasiewicz, “A particle filtering based approach to approximating interactive pomdps,” in *AAAI*, 2005, pp. 969–974.
- [59] M. H. Lim, T. J. Becker, M. J. Kochenderfer, C. J. Tomlin, and Z. N. Sunberg, “Optimality guarantees for particle belief approximation of pomdps,” *Journal of Artificial Intelligence Research*, vol. 77, pp. 1591–1636, 2023.
- [60] Z. N. Sunberg and M. J. Kochenderfer, “Online algorithms for POMDPs with continuous state, action, and observation spaces,” in *Proceedings of the 28th International Conference on Automated Planning and Scheduling (ICAPS)*, 2018, pp. 259–263.
- [61] M. Lauri and R. Ritala, “Planning for robotic exploration based on forward simulation,” *Robotics and Autonomous Systems*, vol. 83, pp. 15–31, 2016.
- [62] S.-K. Kim, O. Salzman, G. Agarwal *et al.*, “PLGRIM: Hierarchical value learning for large-scale exploration in unknown environments,” in *Proceedings of the 31st International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 31, 2021, pp. 652–662.
- [63] S. Brechtel, T. Gindele, and R. Dillmann, “Probabilistic decision-making under uncertainty for autonomous driving using continuous POMDPs,” in *Proceedings of the IEEE International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2014, pp. 392–399.
- [64] H. Bai, D. Hsu, M. J. Kochenderfer, and W. S. Lee, “Unmanned aircraft collision avoidance using continuous-state POMDPs,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2011, pp. 1–8.
- [65] F. Vanegas, D. Campbell, M. Eich, and F. Gonzalez, “UAV based target finding and tracking in GPS-denied and cluttered environments,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 2307–2313.
- [66] Y. Xiao, S. Katt, A. ten Pas, S. Chen, and C. Amato, “Online planning for target object search in clutter under partial observability,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8241–8247.
- [67] J. Pajarinen and V. Kyrki, “Robotic manipulation of multiple objects as a POMDP,” *Artificial Intelligence*, vol. 247, pp. 213–228, 2017.
- [68] J. Pajarinen, J. Lundell, and V. Kyrki, “POMDP planning under object composition uncertainty: Application to robotic manipulation,” *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 41–56, 2023.
- [69] E. A. Hansen and Z. Feng, “Dynamic programming for pomdps using a factored state representation,” in *AIPS*, 2000, pp. 130–139.
- [70] Z. Feng and E. Hansen, “Approximate planning for factored pomdps,” in *Proceedings of the 6th European Conference on Planning*, 2001.
- [71] T. Veiga, M. Spaan, and P. Lima, “Point-based pomdp solving with factored value function approximation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, no. 1, 2014.
- [72] S. R. Eddy, “Hidden markov models,” *Current opinion in structural biology*, vol. 6, no. 3, pp. 361–365, 1996.
- [73] K. Murphy and Y. Weiss, “The factored frontier algorithm for approximate inference in dbns,” *arXiv preprint arXiv:1301.2296*, 2013.
- [74] M. Ghallab, D. Nau, and P. Traverso, *Automated planning and acting*. Cambridge University Press, 2016.
- [75] V. Lifschitz, *Answer set programming*. Springer Cham, 2019, vol. 3.
- [76] M. Richardson and P. Domingos, “Markov logic networks,” *Machine learning*, vol. 62, no. 1, pp. 107–136, 2006.
- [77] M. Qu and J. Tang, “Probabilistic logic neural networks for reasoning,” *Advances in neural information processing systems*, vol. 32, 2019.
- [78] M. Li, Z. Zhang, and B. Song, “Uncertainty-aware neural-symbolic task planning with balanced exploration and execution for domestic service robot,” *Expert Systems with Applications*, p. 132962, 2026.
- [79] S. Sanner and K. Kersting, “Symbolic dynamic programming for first-order pomdps,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, no. 1, 2010, pp. 1140–1146.
- [80] S. Zhang and P. Stone, “Corpp: Commonsense reasoning and probabilistic planning, as applied to dialog with a mobile robot,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 29, no. 1, 2015.
- [81] H. Kurniawati, D. Hsu, W. S. Lee *et al.*, “Sarsop: Efficient point-based pomdp planning by approximating optimally reachable belief spaces,” in *Robotics: Science and systems*, vol. 2008. Zurich, Switzerland, 2008.
- [82] S. A. Serrano, E. Santiago, J. Martinez-Carranza, E. F. Morales, and L. E. Sucar, “Knowledge-based hierarchical pomdps for task planning,” *Journal of Intelligent & Robotic Systems*, vol. 101, no. 4, p. 82, 2021.
- [83] W. Tang, X. He, Y. Huang, Y. Xiao, C. Lu, and P. Cai, “Tru-pomdp: Task planning under uncertainty via tree of hypotheses and open-ended pomdps,” *arXiv preprint arXiv:2506.02860*, 2025.
- [84] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “Hg-dagger: Interactive imitation learning with human experts,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8077–8083.
- [85] A. Mandlekar, D. Xu, R. Martín-Martín, Y. Zhu, L. Fei-Fei, and S. Savarese, “Human-in-the-loop imitation learning using remote teleoperation,” *arXiv preprint arXiv:2012.06733*, 2020.
- [86] P. Scharre, “Introduction to autonomy in weapon systems,” 2015.
- [87] S. Nahavandi, “Trusted autonomy between humans and robots: Toward human-on-the-loop in robotics and autonomous systems,” *IEEE Systems, Man, and Cybernetics Magazine*, vol. 3, no. 1, pp. 10–17, 2017.
- [88] M. R. Endsley, “Automation and situation awareness,” in *Automation and human performance*. CRC Press, 2018, pp. 163–181.
- [89] D. Davies and R. Parasuraman, *The psychology of violence*. Academic Press, New York, 1982, vol. 1.
- [90] D. R. Davies, G. Matthews, R. B. Stammers, and S. J. Westerman, *Human performance: Cognition, stress and individual differences*. Psychology Press, 2013.
- [91] A. Mack, “Inattention blindness: Looking without seeing,” *Current directions in psychological science*, vol. 12, no. 5, pp. 180–184, 2003.
- [92] C. Y. Wong and G. Seet, “Workload, awareness and automation in multiple-robot supervision,” *International Journal of Advanced Robotic Systems*, vol. 14, no. 3, p. 1729881417710463, 2017.
- [93] S. M. Galster, B. A. Knott, and R. D. Brown, “Managing multiple uavs: Are we asking the right questions?” in *Proceedings of the human factors and ergonomics society annual meeting*, vol. 50, no. 3. SAGE Publications Sage CA: Los Angeles, CA, 2006, pp. 545–549.
- [94] J. Baraglia, M. Cakmak, Y. Nagai, R. P. Rao, and M. Asada, “Efficient human-robot collaboration: when should a robot take initiative?” *The International Journal of Robotics Research*, vol. 36, no. 5–7, pp. 563–579, 2017.
- [95] A. Murata, “Ergonomics and cognitive engineering for robot-human cooperation,” in *Proceedings 9th IEEE International Workshop on Robot and Human Interactive Communication. IEEE RO-MAN 2000*. IEEE, 2000, pp. 206–211.
- [96] G. Borragán, H. Slama, M. Bartolomei, and P. Peigneux, “Cognitive fatigue: A time-based resource-sharing account,” *Cortex*, vol. 89, pp. 71–84, 2017.
- [97] A. Rajavenkatanarayanan, V. Kanak, K. Tsiakas, J. Brady, D. Calderon, G. Wylie, and F. Makedon, “Towards a robot-based multimodal framework to assess the impact of fatigue on user behavior and performance: A pilot study,” in *Proceedings of the 12th ACM International Conference on Pervasive Technologies Related to Assistive Environments*, 2019, pp. 493–498.
- [98] S. Chitta, I. Sucan, and S. Cousins, “Moveit![ros topics],” *IEEE robotics & automation magazine*, vol. 19, no. 1, pp. 18–19, 2012.
- [99] Y. Tian, Q. Ye, and D. Doermann, “Yolov12: Attention-centric real-time object detectors,” *arXiv preprint arXiv:2502.12524*, 2025.

APPENDIX

APPENDIX CONTENTS

This appendix provides supplementary materials to support the main text.

- 1) Appendix A. Experiments
 - a) Appendix ?? Prompts for Object Detection and Naming
 - b) Appendix ?? Prompts for Property Probing
 - c) Appendix ?? Prompts for Predicate Generator
 - d) Appendix ?? Prompts for Action Generator
 - e) Appendix ?? Prompts for Initial State Descriptor
 - f) Appendix ?? Prompts for Goal State Descriptor
 - g) Appendix ?? Prompts for Task Planner
 - h) Appendix ?? Prompts for Replanning
- 2) Appendix ?? Definitions of Shapes, Dimension, and Properties
 - a) Appendix ?? Shapes
 - b) Appendix ?? Dimensions
 - c) Appendix ?? Properties
- 3) Appendix ?? Results of Task Planner and Plan Validator
 - a) Appendix ?? Task Planning
 - b) Appendix ?? Plan Validation
- 4) Appendix ?? Experiments
 - a) Appendix ?? Instances
 - b) Appendix ?? Property Probing Execution
 - c) Appendix ?? Task Planning Demo
- 5) Appendix ?? Execution Layer Reproducibility
 - a) Appendix ?? Overview of the PbD Layer
 - b) Appendix ?? Mapping Symbolic Actions to Skills
 - c) Appendix ?? Controller Configuration
 - d) Appendix ?? Success Criteria
 - e) Appendix ?? Abort and Safety Conditions
 - f) Appendix ?? PbD Execution Pseudocode
 - g) Appendix ?? Example Skill Trace

Hi! My name is τ !